# Project Proposal — Proposed Solution

AI WeatherWise API

Project: AI WeatherWise API    Track: AI-ML & GEN-AI Track

Prepared: 07 October 2026    Team ID: Not provided

Maximum Marks: 5

## Team Members & Assigned Roles

| Team Member | Role |
|---|---|
| Jai Bharath A | Team Lead & Lead Backend/API Architect |
| Mukesh Balaji | Generative AI & Prompt Engineering Engineer |
| Yadagiri Mani | Database & Data Engineering Specialist |
| Deepak G | Security, Testing & Reliability Engineer |
| Naveen | API Integration, Documentation & Demonstration Engineer |

## Project Overview

AI WeatherWise API is an intelligent, resilient REST backend that combines live OpenWeatherMap observations with Google Gemini 2.5 Flash to produce human-readable weather summaries and practical recommendations. It also supports secure user accounts and favorite locations in MongoDB.

## Core Solution Flow

**Client → Express API → Weather Service → OpenWeatherMap → normalized weather context → AI Service → Gemini / rule fallback → JSON response. Protected location requests additionally pass through JWT middleware and MongoDB persistence.**

| Module | What it does | Primary owner |
|---|---|---|
| Authentication | Registration, bcrypt hashing, login, JWT profile protection | Jai + Deepak |
| Weather Proxy | Live weather retrieval with mock fallback | Jai |
| Locations | Favorite city CRUD and user isolation | Yadagiri |
| Gen-AI Summary | Plain-English weather interpretation | Mukesh |
| Recommendations | Outfit, commute and hydration advice | Mukesh |
| Testing & Reliability | Regression suite, latency checks and failure testing | Deepak + Naveen |

## Expected Impact

- Reduces interpretation effort for everyday weather decisions.
- Demonstrates practical use of Gen-AI rather than a chatbot-only concept.
- Provides an evaluator-friendly API with clear endpoints and repeatable tests.
- Builds resilience into the architecture instead of treating external APIs as guaranteed.

## Security & Responsible AI Notes

Do not expose API keys to clients. Keep prompts grounded in retrieved weather data, avoid fabricating severe-weather claims, and preserve a deterministic fallback for continuity. Client applications should clearly communicate when a response is based on fallback/mock data.